

Space Invaders lateral: todas as alterações no código

Oct 10, 2026 · @André

Para transformar o Space Invaders vertical numa versão lateral em tela cheia, o Claude alterou 11 dos 13 arquivos do projeto. Este documento lista cada alteração no código, com o trecho antes e depois, e conta o bug dos inimigos de lado que apareceu no caminho. Só entram as mudanças feitas para a versão lateral; o que o grupo já tinha feito (telas, pausa, sprites, barreiras, loop de tempo fixo) é o ponto de partida e não está descrito aqui.

Resumo dos arquivos

Arquivo	O que mudou	Seção
Projetil.js	Tiro deitado, andando em X, velocidade 7 para 9	Tiros, nave e inimigos
Jogador.js	Move em X e Y, diagonal na mesma velocidade, tiro para a direita	Tiros, nave e inimigos
Inimigo.js	Bloco anda em Y; animação pela posição Y	Tiros, nave e inimigos
InimigoRapido.js	<code>mover()</code> em Y	Tiros, nave e inimigos
InimigoAtirador.js	Tiro sai da frente (esquerda) e vai para a esquerda	Tiros, nave e inimigos
Sprites.js	Função nova <code>girar()</code> e canhão virado para a direita	Giro dos desenhos
Barreira.js	Escudo girado, com a abertura para o jogador	Giro dos desenhos
Jogo.js	Tamanho da tela, onda em colunas, escudos em coluna, WASD, colisões, derrota pela borda esquerda	Jogo.js
index.html	Barra do topo, canvas 1280×720, título e instruções	Tela cheia e textos
style.css	Layout que ocupa a janela, canvas sem deformar	Tela cheia e textos
Tela.js	Textos da tela de início	Tela cheia e textos

Arquivo	O que mudou	Seção
Entidade.js	Sem mudanças	
InimigoBasico.js	Sem mudanças (ficou igual ao do grupo depois da correção do bug)	

Tiros, nave e inimigos

Projetoil.js

O tiro passou a viajar na horizontal: ficou deitado (12×4 em vez de 4×12), anda em X e some ao sair pela esquerda ou pela direita. A velocidade subiu de 7 para 9, uma decisão de jogabilidade, porque a tela agora é mais larga. Como a largura da tela muda com a janela, `atualizar()` passou a receber `larguraCanvas`, em vez de usar o 600 fixo.

```
// Antes
class Projetoil extends Entidade {
  constructor(x, y, direcao, cor = '#ffffff') {
    super(x, y, 4, 12, cor);
    this.velocidade = 7;
    this.direcao = direcao; // -1 = sobe (jogador), 1 = desce (inimigo)
  }

  atualizar() {
    this.y += this.velocidade * this.direcao;
    if (this.y < 0 || this.y > 600) {
      this.ativo = false;
    }
  }
}

// Depois
class Projetoil extends Entidade {
  constructor(x, y, direcao, cor = '#ffffff') {
    super(x, y, 12, 4, cor); // largura 12 e altura 4: um "traço" deitado
    this.velocidade = 9;
    this.direcao = direcao; // 1 = direita (jogador), -1 = esquerda (inimigo)
  }

  atualizar(larguraCanvas) {
    this.x += this.velocidade * this.direcao;
    if (this.x + this.largura < 0 || this.x > larguraCanvas) {
```

```

        this.ativo = false;
    }
}

```

Jogador.js

Três mudanças: a caixa da nave ficou mais alta que larga (30×56, porque o desenho do canhão é girado); `mover()` passou a andar em X e Y, com um fator de 0,7071 na diagonal para a nave não andar ~41% mais rápido; e o tiro sai da ponta direita da nave, na altura do meio. O `desenhar()` usa o canhão girado (`Sprites.PARA_DIREITA`).

```

// Antes
super(x, y, 50, 20, '#7CFC00');
...
mover(direcao, larguraCanvas) {
    this.x += this.velocidade * direcao;
    if (this.x < 0) this.x = 0;
    if (this.x + this.largura > larguraCanvas) {
        this.x = larguraCanvas - this.largura;
    }
}

atirar(listaProjeteis) {
    if (this.cooldownTiro > 0) return;
    const centroX = this.x + this.largura / 2 - 2;
    listaProjeteis.push(new Projtil(centroX, this.y - 12, -1, '#7CFC00'));
    this.cooldownTiro = 15;
}

desenhar(ctx) {
    Sprites.desenhar(ctx, Sprites.CANHAO[0], this.x, this.y, this.largura,
this.altura, this.cor);
}

// Depois
super(x, y, 30, 56, '#7CFC00'); // sprite do canhão girado: mais alto que
largo
...
mover(direcaoX, direcaoY, larguraCanvas, alturaCanvas) {
    const fator = direcaoX !== 0 && direcaoY !== 0 ? Math.SQRT1_2 : 1;
    this.x += this.velocidade * direcaoX * fator;
    this.y += this.velocidade * direcaoY * fator;
    this.x = Math.max(0, Math.min(this.x, larguraCanvas - this.largura));
    this.y = Math.max(0, Math.min(this.y, alturaCanvas - this.altura));
}

```

```

}

atirar(listaProjeteis) {
  if (this.cooldownTiro > 0) return;
  const pontaX = this.x + this.largura;
  const meioY = this.y + this.altura / 2 - 2;
  listaProjeteis.push(new Projatil(pontaX, meioY, 1, '#7CFC00'));
  this.cooldownTiro = 15;
}

desenhar(ctx) {
  Sprites.desenhar(ctx, Sprites.PARA_DIREITA.CANHAO[0], this.x, this.y,
  this.largura, this.altura, this.cor);
}

```

Inimigo.js, InimigoRapido.js e InimigoAtirador.js

O bloco de inimigos agora sobe e desce (a direção 1 significa "para baixo" e -1 "para cima"). Por isso `mover()` anda em Y, não precisa mais da largura do canvas, e a animação de 2 quadros troca conforme o Y. O `InimigoRapido` sobrescreve `mover()` do mesmo jeito, com o dobro da velocidade. O `InimigoAtirador` sobrescreve `tentarAtirar()`: o tiro sai da frente (lado esquerdo) e vai para a esquerda. A herança e o polimorfismo continuam iguais.

```

// Inimigo.js - antes
this.direcao = 1; // 1 = direita, -1 = esquerda
...
quadroAtual() {
  return Math.floor(Math.abs(this.x) / 10) % 2;
}
mover(velocidadeBase, larguraCanvas, alturaQueda) {
  this.x += velocidadeBase * this.direcao;
}

// Inimigo.js - depois
this.direcao = 1; // 1 = para baixo, -1 = para cima
...
quadroAtual() {
  return Math.floor(Math.abs(this.y) / 10) % 2;
}
mover(velocidadeBase) {
  this.y += velocidadeBase * this.direcao;
}

```

```
// InimigoRapido.js - antes
mover(velocidadeBase, larguraCanvas) {
  this.x += velocidadeBase * this.multiplicadorVelocidade * this.direcao;
}

// InimigoRapido.js - depois
mover(velocidadeBase) {
  this.y += velocidadeBase * this.multiplicadorVelocidade * this.direcao;
}
```

```
// InimigoAtirador.js - antes
tentarAtirar(listaProjeteis) {
  if (Math.random() < this.chanceDeAtirar) {
    const centroX = this.x + this.largura / 2 - 2;
    listaProjeteis.push(new Projtil(centroX, this.y + this.altura, 1,
    '#c266ff'));
  }
}

// InimigoAtirador.js - depois
tentarAtirar(listaProjeteis) {
  if (Math.random() < this.chanceDeAtirar) {
    const frenteX = this.x - 12;
    const meioY = this.y + this.altura / 2 - 2;
    listaProjeteis.push(new Projtil(frenteX, meioY, -1, '#c266ff'));
  }
}
```

Giro dos desenhos

Sprites.js

Os desenhos são listas de linhas de texto (# pinta, . deixa vazio). A função nova `Sprites.girar(mapa, sentido)` gira uma dessas listas em 90°: 'horario' leva o topo para a direita e 'anti' leva o topo para a esquerda. Ela não altera o desenho original, devolve um novo. `Sprites.PARA_DIREITA.CANHAO` guarda o canhão já girado, calculado uma só vez ao carregar a página, para o jogo não recalculá-lo a cada quadro.

```
// Antes: o arquivo terminava em }; (fim do objeto Sprites)

// Depois: foi acrescentado no final
Sprites.girar = function (mapa, sentido) {
  const linhas = mapa.length;
```

```

const colunas = mapa[0].length;
const novo = [];
for (let i = 0; i < colunas; i++) {
  let linha = '';
  for (let j = 0; j < linhas; j++) {
    linha += sentido === 'horario' ? mapa[linhas - 1 - j][i] : mapa[j]
[colunas - 1 - i];
  }
  novo.push(linha);
}
return novo;
};

Sprites.PARA_DIREITA = {
  CANHAO: Sprites.CANHAO.map((quadro) => Sprites.girar(quadro, 'horario')),
};

```

Barreira.js

O desenho do escudo tinha a "boca" embaixo, de frente para o jogador. Como o jogador agora fica à esquerda, o desenho é girado no sentido horário dentro de `criarEscudo()`, e a boca passa a ficar do lado esquerdo. O escudo, que tinha 10 colunas e 6 linhas, passa a ter 6 colunas e 10 linhas de peças.

```

// Antes
static criarEscudo(xInicial, yInicial) {
  const formato = [
    '..#####..',
    '.#####.',
    '#####',
    '#####',
    '###....###',
    '##.....##',
  ];
  ...
}

// Depois
static criarEscudo(xInicial, yInicial) {
  const formatoOriginal = [
    '..#####..',
    '.#####.',
    '#####',
    '#####',
    '###....###',
  ];

```

```

    '##.....##',
  ];
  const formato = Sprites.girar(formatoOriginal, 'horario');
  ...
}

```

Jogo.js

É o arquivo com mais mudanças, porque é onde ficam as regras e o tamanho da tela. A estrutura do loop (passo fixo de 60 Hz, pausa, início e fim) não foi mexida.

Método ou trecho	O que mudou
<code>constructor</code>	Guarda a caixa <code>.area-jogo</code> , chama <code>_ajustarTamanho()</code> antes de montar o jogo e ouve o evento <code>resize</code> da janela
<code>_ajustarTamanho()</code> (novo)	Define a altura do mundo em 720 e a largura pela proporção da janela, entre 960 e 1680
<code>_resetarEstado()</code>	Nave nasce à esquerda ($x = 40$), centralizada na altura; o bloco de inimigos começa descendo
<code>iniciar()</code>	Chama <code>_ajustarTamanho()</code> antes de cada partida
<code>_configurarTeclado()</code>	Setas para cima e para baixo também não rolam a página
<code>_criarOnda()</code>	4 colunas de 8 inimigos, em vez de 4 linhas de 8, centralizadas na altura e coladas à direita
<code>_criarBarreiras()</code>	4 escudos em coluna vertical ($x = 130$), espaçados pela altura
<code>_tratarInput()</code>	Setas ou WASD nos 4 sentidos
<code>_atualizarInimigos()</code>	Bate no topo ou no chão em vez das bordas laterais; ao bater, o bloco avança 48px para a esquerda
<code>_atualizarProjeteis()</code>	Passa a largura da tela para <code>atualizar()</code>
<code>_verificarColisoes()</code>	Tiro do jogador atravessa os escudos; choque entre inimigo e nave tira uma vida
<code>_verificarFimDeJogo()</code>	Derrota quando um invasor chega à borda esquerda

Método ou trecho	O que mudou
Constantes (fim do arquivo)	ALTURA_MUNDO, LARGURA_MINIMA, LARGURA_MAXIMA e AVANCO_DO_BLOCO

Tamanho da tela

A altura é fixa (720) e só a largura muda, para a dificuldade ser igual em qualquer monitor. Com o jogo parado, mudar o tamanho da janela refaz o cenário; durante a partida o CSS só escala a imagem, sem deformar, e o novo tamanho vale na partida seguinte.

```
// Novo
_ajustarTamanho() {
  const proporcao = this.areaJogo.clientWidth / this.areaJogo.clientHeight;
  const largura = Math.round(Jogo.ALTURA_MUNDO * proporcao);
  this.canvas.height = Jogo.ALTURA_MUNDO;
  this.canvas.width = Math.min(Math.max(largura, Jogo.LARGURA_MINIMA),
Jogo.LARGURA_MAXIMA);
}

// No constructor
window.addEventListener('resize', () => {
  if (this.rodando) return;
  this._ajustarTamanho();
  this._resetarEstado();
  this._desenhar();
});

// No fim do arquivo
Jogo.ALTURA_MUNDO = 720;
Jogo.LARGURA_MINIMA = 960;
Jogo.LARGURA_MAXIMA = 1680;
Jogo.AVANCO_DO_BLOCO = 48;
```

Onda de inimigos e escudos

```
// Antes: _criarOnda()
const linhas = [
  { tipo: InimigoAtirador, y: 40 },
  { tipo: InimigoRapido, y: 90 },
  { tipo: InimigoBasico, y: 140 },
  { tipo: InimigoBasico, y: 190 },
];
linhas.forEach((linha) => {
  for (let i = 0; i < 8; i++) {
```

```

    const x = 60 + i * 80;
    this.inimigos.push(new linha.tipo(x, linha.y));
  }
});

// Depois: _criarOnda()
const colunas = [
  { tipo: InimigoAtirador },
  { tipo: InimigoRapido },
  { tipo: InimigoBasico },
  { tipo: InimigoBasico },
];
const inimigosPorColuna = 8;
const espacoVertical = 72;
const espacoHorizontal = 64;
const alturaBloco = (inimigosPorColuna - 1) * espacoVertical + 24;
const yInicial = (this.canvas.height - alturaBloco) / 2;

colunas.forEach((coluna, c) => {
  const x = this.canvas.width - 90 - c * espacoHorizontal;
  for (let i = 0; i < inimigosPorColuna; i++) {
    const y = yInicial + i * espacoVertical;
    this.inimigos.push(new coluna.tipo(x, y));
  }
});

```

A coluna do atirador (30 pontos) fica na direita, a mais distante da nave, como a linha de cima ficava no jogo original.

```

// Antes: _criarBarreiras()
const larguraEscudo = 10 * Barreira.TAMANHO;
const espaco = this.canvas.width / quantidade;
for (let i = 0; i < quantidade; i++) {
  const x = espaco * i + (espaco - larguraEscudo) / 2;
  pedacos = pedacos.concat(Barreira.criarEscudo(x, 460));
}

// Depois: _criarBarreiras()
const alturaEscudo = 10 * Barreira.TAMANHO;
const espaco = this.canvas.height / quantidade;
const x = 130;
for (let i = 0; i < quantidade; i++) {
  const y = espaco * i + (espaco - alturaEscudo) / 2;
  pedacos = pedacos.concat(Barreira.criarEscudo(x, y));
}

```

Controles e movimento do bloco

```
// Antes: _tratarInput()
if (this.teclas['ArrowLeft']) this.jogador.mover(-1, this.canvas.width);
if (this.teclas['ArrowRight']) this.jogador.mover(1, this.canvas.width);
if (this.teclas['Space']) this.jogador.atirar(this.projeteisJogador);

// Depois: _tratarInput()
const t = this.teclas;
const direcaoX = (t['ArrowRight'] || t['KeyD'] ? 1 : 0) - (t['ArrowLeft'] ||
t['KeyA'] ? 1 : 0);
const direcaoY = (t['ArrowDown'] || t['KeyS'] ? 1 : 0) - (t['ArrowUp'] ||
t['KeyW'] ? 1 : 0);
if (direcaoX !== 0 || direcaoY !== 0) {
  this.jogador.mover(direcaoX, direcaoY, this.canvas.width,
this.canvas.height);
}
if (t['Space']) this.jogador.atirar(this.projeteisJogador);
```

Cada eixo vale 1, -1 ou 0 (tecla da direção positiva menos a da negativa), o que também resolve apertar duas teclas opostas ao mesmo tempo: elas se anulam.

```
// Antes: _atualizarInimigos()
inimigo.mover(this.velocidadeBaseInimigos, this.canvas.width);
...
if (inimigo.x <= 0 || inimigo.x + inimigo.largura >= this.canvas.width) {
  bateuNaBorda = true;
}
...
if (bateuNaBorda) {
  this.direcaoBloco *= -1;
  this.inimigos.forEach((inimigo) => (inimigo.y += 20));
}

// Depois: _atualizarInimigos()
inimigo.mover(this.velocidadeBaseInimigos);
...
if (inimigo.y <= 0 || inimigo.y + inimigo.altura >= this.canvas.height) {
  bateuNaBorda = true;
}
...
if (bateuNaBorda) {
  this.direcaoBloco *= -1;
  this.inimigos.forEach((inimigo) => (inimigo.x -= Jogo.AVANCO_DO_BLOCO));
}
```

Colisões e fim de jogo

Três decisões de regra estão aqui. O tiro do jogador atravessa os próprios escudos, porque eles ficam à frente da nave e, sem isso, ela não conseguiria atirar. Bater de frente num invasor tira uma vida e destrói o invasor, porque a nave agora anda pela tela toda. E a derrota acontece quando um invasor chega à borda esquerda.

```
// Antes: escudos recebiam tiros dos dois lados
[...this.projeteisJogador, ...this.projeteisInimigo].forEach((projetil) => {
  ... });

// Depois: só os tiros dos inimigos
this.projeteisInimigo.forEach((projetil) => { ... });

// Novo, em _verificarColisoos()
this.inimigos.forEach((inimigo) => {
  if (inimigo.ativo && inimigo.colideCom(this.jogador)) {
    inimigo.ativo = false;
    this.jogador.vidas--;
  }
});

// Antes: _verificarFimDeJogo()
} else if (this.inimigos.some((i) => i.y + i.altura >= this.jogador.y)) {
  this._finalizar(false, 'Os invasores chegaram até você!');
}

// Depois
} else if (this.inimigos.some((i) => i.x <= 0)) {
  this._finalizar(false, 'Os invasores passaram pela sua defesa!');
}
```

Tela cheia e textos

index.html

O título e as instruções foram para uma barra fina no topo (`<header class="topo">`), para sobrar o máximo de espaço para o jogo. O canvas passou de 800×600 para 1280×720 como valor inicial; o tamanho real é definido pelo `Jogo.js` ao abrir a página.

```

<!-- Antes -->
<title>Space Invaders - P00</title>
...
<h1>Space Invaders</h1>
<p class="instrucoes">Setas ← → para mover | Espaço para atirar | P ou Esc
para pausar</p>
...
<canvas id="canvas" width="800" height="600"></canvas>

<!-- Depois -->
<title>Space Invaders Lateral - P00</title>
...
<header class="topo">
  <h1>Space Invaders <small>lateral</small></h1>
  <p class="instrucoes">Setas ou WASD para mover | Espaço para atirar | P ou
Esc para pausar</p>
</header>
...
<canvas id="canvas" width="1280" height="720"></canvas>

```

style.css

A página antes era um bloco de 804px centralizado. Agora é uma coluna que ocupa a janela inteira (topo, área do jogo que cresce, placar). A área do jogo usa `flex: 1` para pegar todo o espaço livre, e o canvas preenche essa área com `object-fit: contain`, que escala o desenho sem deformá-lo se a janela mudar de formato no meio da partida. O `overflow: hidden` garante que nunca aparece barra de rolagem.

```

/* Antes */
body { display: flex; justify-content: center; align-items: center; min-height: 100vh; }
.tela { text-align: center; }
.area-jogo { position: relative; width: 804px; margin: 0 auto; border: 2px solid #7CFC00; }
canvas { background: #000; display: block; }
#hud { display: flex; justify-content: space-between; width: 804px; margin: 8px auto 0; font-size: 18px; }

/* Depois */
html, body { height: 100%; }
body { overflow: hidden; }
.tela { height: 100%; display: flex; flex-direction: column; padding: 8px 12px; }
.topo { display: flex; justify-content: space-between; align-items: baseline; gap: 16px; margin-bottom: 8px; }
.area-jogo { position: relative; flex: 1; min-height: 0; border: 2px solid #7CFC00; background: #000; }
canvas { display: block; width: 100%; height: 100%; object-fit: contain; }
#hud { display: flex; justify-content: space-between; align-items: center; margin-top: 8px; font-size: 18px; }

```

Também foram ajustados o tamanho do `h1` (22px), o estilo do `h1 small` e a remoção das margens que não fazem mais sentido. Os estilos da tela de início, do botão e da legenda dos inimigos não mudaram.

Tela.js

Só o texto da tela de início mudou: o título e os controles (setas ou WASD, nos 4 sentidos) e uma frase com o objetivo.

```

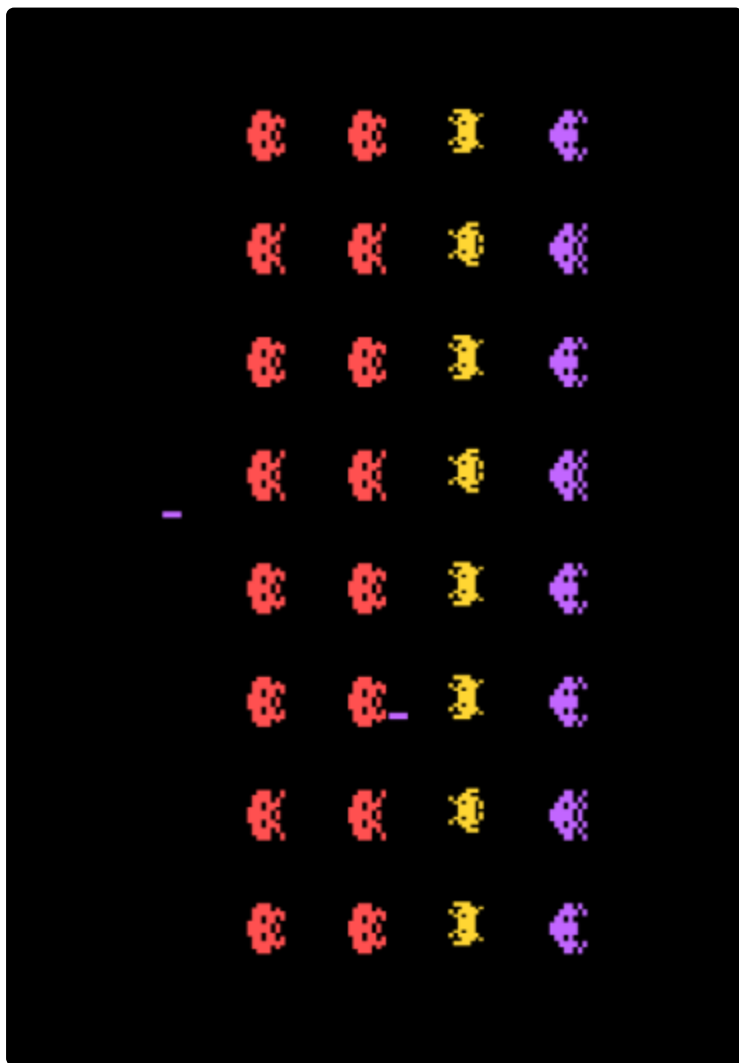
// Antes
this.titulo.textContent = 'SPACE INVADERS';
this.texto.innerHTML =
  'Use as setas ← → para mover e a barra de espaço para atirar.<br><br>' +
  ...

// Depois
this.titulo.textContent = 'SPACE INVADERS LATERAL';
this.texto.innerHTML =
  'Use as setas (ou W A S D) para mover nos 4 sentidos e a barra de espaço para atirar.<br>' +
  'Destrua os invasores antes que passem pela sua defesa.<br><br>' + ...

```

Bug: inimigos de lado

O que aconteceu. Na primeira versão lateral, o Claude girou os desenhos dos três tipos de inimigo (polvo, caranguejo e lula) 90° para a esquerda, junto com o do canhão, achando que eles deveriam "olhar" para a nave. O resultado foi que os invasores apareciam deitados, diferente do jogo original. O André percebeu no print e pediu que ficassem na horizontal, em pé.



Causa. O giro foi aplicado em três lugares ao mesmo tempo: em `Sprites.js` (um grupo `PARA_ESQUERDA` com os três desenhos girados), nos construtores das três subclasses (que usavam `Sprites.PARA_ESQUERDA`) e nas caixas de colisão, que foram trocadas de largura×altura (por exemplo 32×24 virou 24×32).

```
// Versão com o bug
// Sprites.js
Sprites.PARA_ESQUERDA = {
  LULA: girarTodos(Sprites.LULA, 'anti'),
  CARANGUEJO: girarTodos(Sprites.CARANGUEJO, 'anti'),
  POLVO: girarTodos(Sprites.POLVO, 'anti'),
};

// InimigoBasico.js
super(x, y, 24, 32, '#ff5050', 10);
this.quadros = Sprites.PARA_ESQUERDA.POLVO;

// InimigoRapido.js
super(x, y, 22, 28, '#ffd633', 20);
this.quadros = Sprites.PARA_ESQUERDA.CARANGUEJO;

// InimigoAtirador.js
super(x, y, 24, 32, '#c266ff', 30);
this.quadros = Sprites.PARA_ESQUERDA.LULA;
```

Correção. O grupo `PARA_ESQUERDA` foi apagado, as três subclasses voltaram a usar os desenhos originais (`Sprites.POLVO`, `Sprites.CARANGUEJO`, `Sprites.LULA`) e as caixas de colisão voltaram ao tamanho original. O giro ficou só no canhão e nos escudos. Em `Jogo.js`, a altura usada para centralizar o bloco na tela também voltou de 32 para 24, que é a altura dos inimigos.

```
// Versão corrigida
// InimigoBasico.js
super(x, y, 32, 24, '#ff5050', 10);
this.quadros = Sprites.POLVO;

// InimigoRapido.js
super(x, y, 28, 22, '#ffd633', 20);
this.quadros = Sprites.CARANGUEJO;

// InimigoAtirador.js
super(x, y, 32, 24, '#c266ff', 30);
this.quadros = Sprites.LULA;

// Sprites.js: só o canhão ficou girado
Sprites.PARA_DIREITA = {
  CANHAO: Sprites.CANHAO.map((quadro) => Sprites.girar(quadro, 'horario')),
};
```

Depois da correção, o jogo foi testado de novo no navegador e todas as verificações passaram. Por isso `InimigoBasico.js` e as caixas de colisão dos inimigos ficaram iguais às do grupo.



Como rodar e como foi testado

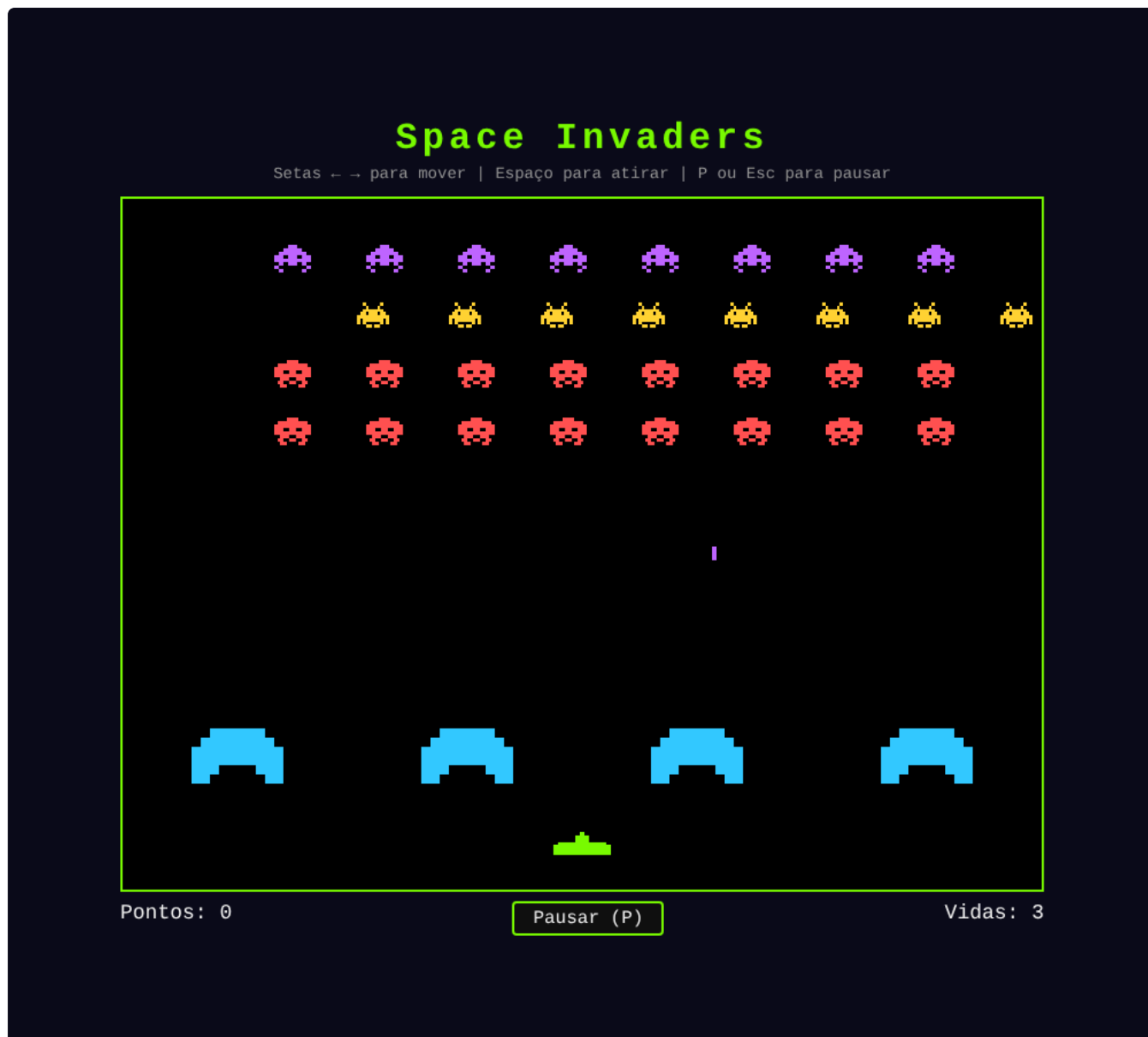
Como rodar. Extrair o `space-invaders-lateral.zip` e abrir o `index.html` no navegador, com a pasta `js/` ao lado dele. Não precisa instalar nada. Controles: setas ou WASD para mover, espaço para atirar, P ou Esc para pausar, Enter para iniciar e reiniciar.

Como foi testado. No navegador (Chromium automatizado), com a janela em 1600×900, 900×900, 1280×720 e 2200×900. Todas as verificações passaram e não houve erro no console.

Verificação	Resultado
Tamanho da tela	Altura 720; largura 1399 em 1600×900 e 1680 em tela ultrawide; sem barra de rolagem
Movimento	Quatro setas, W e D funcionam; a nave não sai da tela
Diagonal	180px em 30 passos, igual aos outros sentidos
Tiro do jogador	Vai para a direita, atravessa o escudo e acerta o inimigo (+10 pontos)

Verificação	Resultado
Tiro dos inimigos	Atiradores atiram para a esquerda; o escudo segura o tiro (resistência 3 para 2); o tiro na nave tira 1 vida
Bloco de inimigos	Sobe e desce, avança para a esquerda e fica entre o topo e o chão
Choque	Bater num inimigo tira 1 vida e destrói o inimigo
Fim de jogo	Derrota ao chegar à borda esquerda; vitória ao destruir todos; Enter reinicia com 3 vidas e 32 inimigos
Pausa	P pausa e congela os inimigos; P de novo continua
Janela	Mudar o tamanho no meio da partida não derruba o jogo; com o jogo parado, a largura se ajusta

Antes, a versão vertical do grupo:



Depois, a versão lateral em partida:



Tela de início da versão lateral:

